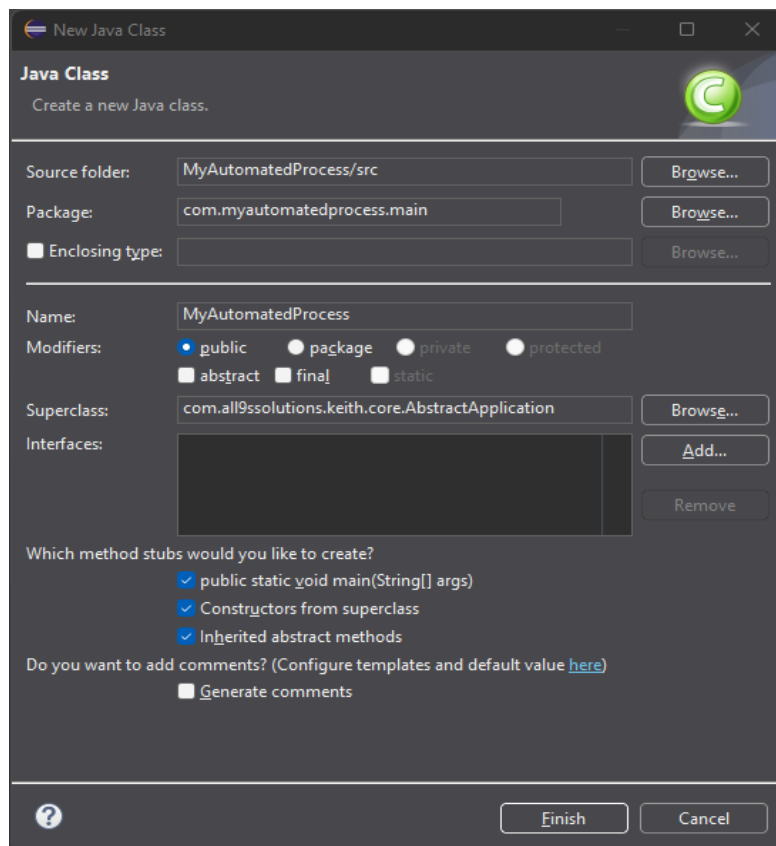
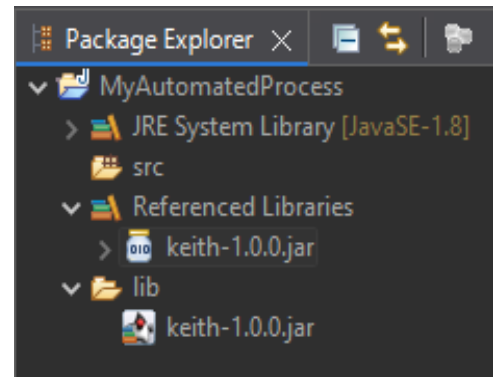


Keith Framework

Implementation

Since the Keith framework encloses code in exception-handling logic and enables Java util logging, programmers have the freedom to build without writing boilerplate structures. But how difficult is it to incorporate the Keith framework?

First, create a new Java project and add the keith-[version].jar, where [version] is replaced with the jar version such as 1.0.0.



Second, create a new class extending `com.all9ssolutions.keith.core.AbstractApplication`, and check the options to create method stubs for the main method, constructors from superclass, and inherited abstract methods. By selecting these options the IDE will generate code required by the Keith framework.

Keith Framework

Using the names provided, your generated code should look something like the following:

```
package com.myautomatedprocess.main;

import java.util.Map;
import com.all9ssolutions.keith.core.AbstractApplication;

public class MyAutomatedProcess extends AbstractApplication {
    private static final long serialVersionUID = 1L;

    public MyAutomatedProcess(String[] args) {
        super(args);
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
    }

    @Override
    protected void run() throws Exception {
        // TODO Auto-generated method stub
    }

    @Override
    protected Map<String, String> getProperties() {
        // TODO Auto-generated method stub
        return null;
    }
}
```

Third, connect the logic flow to the superclass by adding a statement to the main() method instantiating your class, and add any properties to use in your logic flow with the getProperties() method:

```
public static void main(String[] args) {
    new MyAutomatedProcess(args);
}

@Override
protected Map<String, String> getProperties() {
    Map<String, String> properties = new HashMap<>();
    properties.put("check.period", "3000"); // in milliseconds
    return properties;
}
```

Keith Framework

Finally, use the run() method to provide logic for the automated process. In the following code, we demonstrate printing the property added via the getProperties() method:

```
@Override
protected void run() throws Exception {
    getApplicationProperties().list(System.out);
}
```

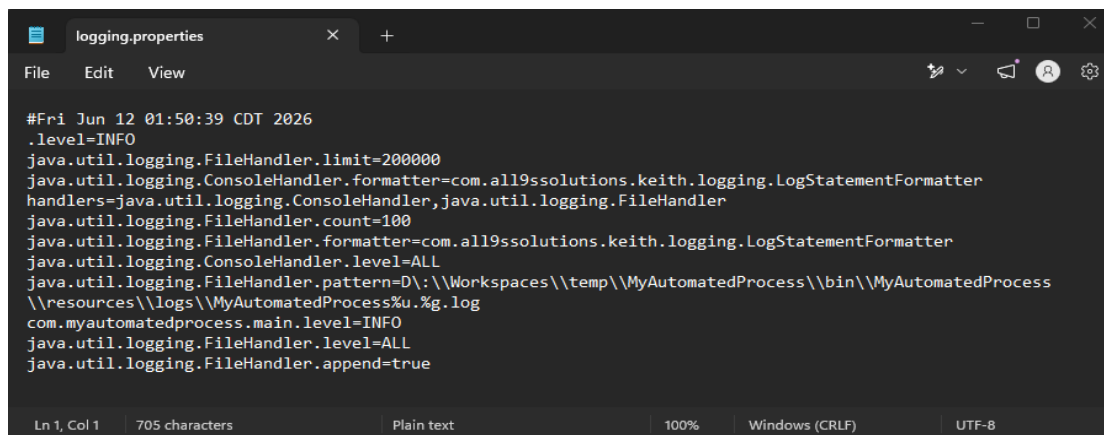
Modifiable Properties

The fundamental principal in the Keith framework is supporting configured properties and exposing them to team members for manipulation at before or during runtime. When an application starts, the Keith framework verifies the external properties file exists and the configured keys are present. **The values are not overwritten insuring they retain any modifications.** The application.properties file is found alongside the compiled automated process. For example, if an application, named *MyAutomatedProcess.jar* process lives on the Desktop, then the external properties will exist in */home/Desktop/MyAutomatedProcess/resources/*.

Example 1

The logging level for an application needs to be raised without restarting the process:

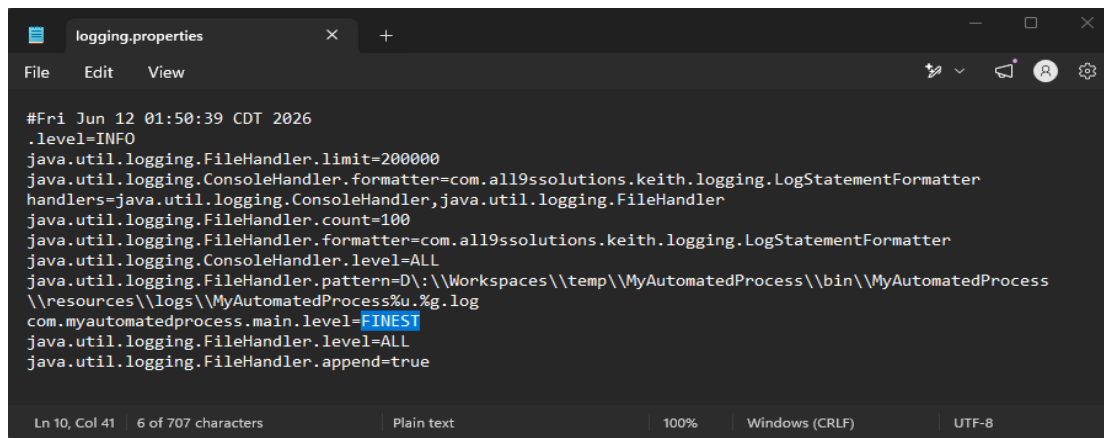
- Open the external logging.properties file.
- Locate the package logging level or add a class level logger key and value.
- Set the appropriate logging level.
- Save the file.

A screenshot of a text editor window titled 'logging.properties'. The editor shows the following content:

```
#Fri Jun 12 01:50:39 CDT 2026
.level=INFO
java.util.logging.FileHandler.limit=200000
java.util.logging.ConsoleHandler.formatter=com.all9ssolutions.keith.logging.LogStatementFormatter
handlers=java.util.logging.ConsoleHandler,java.util.logging.FileHandler
java.util.logging.FileHandler.count=100
java.util.logging.FileHandler.formatter=com.all9ssolutions.keith.logging.LogStatementFormatter
java.util.logging.ConsoleHandler.level=ALL
java.util.logging.FileHandler.pattern=D:\\Workspaces\\temp\\MyAutomatedProcess\\bin\\MyAutomatedProcess
\\resources\\logs\\MyAutomatedProcess%.%g.log
com.myautomatedprocess.main.level=INFO
java.util.logging.FileHandler.level=ALL
java.util.logging.FileHandler.append=true
```

The status bar at the bottom indicates 'Ln 1, Col 1', '705 characters', 'Plain text', '100%', 'Windows (CRLF)', and 'UTF-8'.

Keith Framework



```
#Fri Jun 12 01:50:39 CDT 2026
.level=INFO
java.util.logging.FileHandler.limit=200000
java.util.logging.ConsoleHandler.formatter=com.all9ssolutions.keith.logging.LogStatementFormatter
handlers=java.util.logging.ConsoleHandler,java.util.logging.FileHandler
java.util.logging.FileHandler.count=100
java.util.logging.FileHandler.formatter=com.all9ssolutions.keith.logging.LogStatementFormatter
java.util.logging.ConsoleHandler.level=ALL
java.util.logging.FileHandler.pattern=D:\\Workspaces\\temp\\MyAutomatedProcess\\bin\\MyAutomatedProcess
\\resources\\logs\\MyAutomatedProcess\\u.%g.log
com.myautomatedprocess.main.level=FINEST
java.util.logging.FileHandler.level=ALL
java.util.logging.FileHandler.append=true
```

Example 2

A team member needs to manually shutdown an automated process.

- During development add a property named run and set its initial value to true.
- At convenient stop points in the code, test the value of the run property. If false, then fork the logic to allow shutdown.
- Test the operation:
 - Run the code.
 - Open the external application.properties file.
 - Locate the run property and change it.
 - Save the file.

@Override

```
protected void run() throws Exception {
    boolean run = true;
    while (run) {
        try {
            TimeUnit.SECONDS.sleep(5L);
        } catch (InterruptedException e) {
            e.printStackTrace(System.err);
        }
        System.out.println("Run is: " + run);
        run = Boolean.parseBoolean(getApplicationProperties()
            .getProperty("run", "false"));
    }
    System.out.println("Done!");
}
```

@Override

```
protected Map<String, String> getProperties() {
```

Keith Framework

```
Map<String, String> properties = new HashMap<>();  
properties.put("check.period", "5000");  
properties.put("run", "true");  
return properties;  
}
```

Note:

The code must query the property at each iteration for a changed value. It must access the `AbstractApplication.getApplicationProperties()` method for updates, NOT the `getProperties()` method found in the implementing class.

Keith Framework

Additional Methods

The run() method is required to be overridden and should contain the application logic. But there are additional methods available which may be overridden and are called by the Keith framework:

- Include a preprocess() method to provide logic which should be executed before the Keith framework is fully initialized.
- Include a process() method to provide logic which should be executed after the Keith framework is initialized, but before the run() method is called.
- Include a postprocess() method to provide shut down, or clean up logic, called after the run() method completes.

The Keith framework also insures a logging.properties file exists. In the rare scenario logging levels may need to be raised or lowered, this file is available to receive modifications. To change the log levels for one or more packages beyond a single cycle, modify the external properties file (see Modifiable Properties for more).

Similar behavior may be incorporated in the code. Use the setLogLevel() method in LoggingProperties to change the levels for all application packages. Those levels will be changed for one check period, then return to values set in the logging.properties file.